Hands on Labs

Lab 1

Hands-on lab for assigning limited sudo privileges

- Objective: Create some users and assign them different levels of privileges; To use the AlmaLinux 9 virtual machine

1. Log in to the AlmaLinux virtual machine and create user accounts for Lionel, Katelyn, and Maggie:
   - sudo useradd lionel
   - sudo useradd katelyn
   - sudo useradd Maggie
   - sudo passwd lionel
   - sudo passwd katelyn
   - sudo passwd Maggie
2. Open visudo:
   - sudo visudo

Hands-on lab for assigning limited sudo privileges

- Objective: Create some users and assign them different levels of privileges; To use the AlmaLinux 9 virtual machine

3. Find the STORAGE command alias and remove the comment symbol from in front of it.

4. Add the following lines to the end of the file, using tabs to separate the columns:

```
lionel ALL=(ALL) ALL
katelyn ALL=(ALL) /usr/bin/systemctl status sshd
maggie ALL=(ALL) STORAGE
```

- Save the file and exit visudo.

Hands-on lab for assigning limited sudo privileges

- Objective: Create some users and assign them different levels of privileges; To use the AlmaLinux 9 virtual machine

5. use **su** to log in to the different user accounts. First, log in to Lionel's account and verify that he has full sudo privileges by running several root-level commands:

su – lionel

sudo su –

exit

sudo systemctl status sshd

sudo fdisk -l

exit

Hands-on lab for assigning limited sudo privileges

- Objective: Create some users and assign them different levels of privileges; To use the AlmaLinux 9 virtual machine

6. log in as Katelyn and try to run some root-level commands.

    su – katelyn

    sudo su –

    sudo systemctl status sshd

    sudo systemctl restart sshd

    sudo fdisk -l

    exit

Hands-on lab for assigning limited sudo privileges

- Objective: Create some users and assign them different levels of privileges; To use the AlmaLinux 9 virtual machine

7. Log in as Maggie, and run the same set of commands that you ran for Katelyn.

8. You can set more users for this lab by setting them up in user aliases or Linux groups.

Lab 2

Hands-on lab for disabling the sudo timer

- Objective: disable the sudo timer on your AlmaLinux VM

1. Log in to the same AlmaLinux virtual machine that you used for the previous lab
2. At your own user account command prompt, enter the following commands

    sudo fdisk -l

    sudo systemctl status sshd

    sudo iptables –L

3. At your own user account command prompt, run the following:

Hands-on lab for disabling the sudo timer

- Objective: disable the sudo timer on your AlmaLinux VM

3. At your own user account command prompt, run the following:

    sudo fdisk -l

    sudo -k

    sudo fdisk –l

4. Note how the **sudo -k** command resets your timer, so you will have to enter your password again. Open visudo with the following command:

    sudo visudo

Hands-on lab for disabling the sudo timer

- Objective: disable the sudo timer on your AlmaLinux VM

5. In the Defaults specification section of the file, add the following line:

```
Defaults timestamp_timeout = 0
```

Save the file and exit visudo.

6. Perform the commands that you performed in step 2. This time, you should see that you have to enter a password every time.

7. Open visudo and modify the line that you added so that it looks like this, Save the file and exit visudo:

```
Defaults:lionel timestamp_timeout = 0
```

Hands-on lab for disabling the sudo timer

- Objective: disable the sudo timer on your AlmaLinux VM

6. From your own account shell, repeat the commands that you performed in step 2. Then, log

in as Lionel and perform the commands again.

7. View your own sudo privileges by running the following:

```
sudo -l
```

Lab 3

Hands-on lab for creating an encrypted home directory with adduser

- **Ubuntu 22.04 VM**:

- Install the ecryptfs-utils package
  - sudo apt install ecryptfs-utils

- Create a user account with an encrypted home directory for Cleopatra and then view the results:
  - sudo adduser --encrypt-home cleopatra
  - ls -l /home

- Log in as Cleopatra and run the ecryptfs-unwrap-passphrase command:
  - su - cleopatra
  - ecryptfs-unwrap-passphrase
  - exit

Lab 4

Hands-on lab for setting password complexity criteria

- CentOS, AlmaLinux, or Ubuntu virtual machine

1. For Ubuntu only, install the libpam-pwquality package:
   - sudo apt install libpam-pwquality
2. Open the /etc/security/pwquality.conf file in your preferred text editor. Remove the comment symbol from in front of the minlen line and change the value to 19. It should now look like this:
   - minlen = 19
3. Save the file and exit the editor.
4. Create a user account for Goldie and attempt to assign her the passwords turkeylips, TurkeyLips, and Turkey93Lips. Note the change in each warning message.
5. In the pwquality.conf file, comment out the minlen line. Uncomment the minclass line and the maxclassrepeat line. Change the maxclassrepeat value to 5. The lines should now look like this:
   - minclass = 3
   - maxclassrepeat = 5
6. Save the file and exit the text editor.
7. Try assigning various passwords that do not meet the complexity criteria that you've set to Goldie's account and view the results.

Lab 5

Hands-on lab for setting account and password expiry data

1. On your CentOS or AlmaLinux VM, create a user account for Samson with the expiration date of June 30, 2025, and view the results:
   - sudo useradd -e 2025-06-30 samson
   - sudo chage -l samson
2. For Ubuntu, run these commands:
   - sudo useradd -m -d /home/samson -s /bin/bash -e 2025-06-30 samson
   - sudo chage -l samson
3. Use usermod to change Samson’s account expiration date to July 31, 2025:
   - sudo usermod -e 2025-07-31 samson
   - sudo chage -l samson
4. Assign a password to Samson’s account, then force him to change his password on his first login. Log in as Samson, change his password, then log in to your own account:
   - sudo passwd samson
   - sudo passwd -e samson
   - sudo chage -l samson
   - su - samson
   - exit

Hands-on lab for setting account and password expiry data

5. Use chage to set a five-day waiting period for changing passwords, a password expiration period of 90 days, an inactivity period of two days, and a warning period of five days:

- sudo chage -m 5 -M 90 -I 2 -W 5 samson
- sudo chage -l samson

Lab 6

Hands-on lab for detecting compromised passwords

- In this lab, you'll use the pwnedpasswords API in order to check your own passwords.

1. Use curl to see how many passwords there are with the 21BD1 string in their password hashes:

   curl https://api.pwnedpasswords.com/range/21BD1

2. In the home directory of any of your Linux virtual machines, paste the pwnpassword.sh script provided by the lecturer

3. Add the executable permission to the script:

   chmod u+x pwnedpasswords.sh

4. Run the script, specifying TurkeyLips as a password:

   ./pwnedpasswords.sh TurkeyLips

5. Repeat Step 4 as many times as you like, using a different password each time.

Lab 7

Hands-on lab for configuring pam_tally2

- CentOS 7
- 1. Open the /etc/pam.d/login file for editing. Look for the line that invokes the pam_securetty module. (That should be around line 2.) Beneath that line, insert this line:

auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200

- 2. Save the file and exit the editor.
- 3. Place the same line at the top of the /etc/pam.d/password.auth and /etc/pam.d/system.auth files, just above the first auth required line. (The comment at the top of these files says to not hand-edit them because running authconfig will destroy the edits. Unfortunately, you have to hand edit them, because authconfig won’t configure this for you.)

Hands-on lab for configuring pam_tally2

- 4. For this step, you'll need to log out of your own account because pam_tally2 doesn't work with su. So log out and, while purposely using the wrong password, attempt to log in to the samson account that you created in the previous lab. Keep doing that until you see a message saying that the account is locked. Note that when the deny value is set to 4, it will actually take five failed login attempts to lock Samson out.
- 5. Log back in to your own user account. Run this command and note the output:

  sudo pam_tally2
- 6. For this step, you'll simulate that you're a help desk worker, and Samson has just called to request that you unlock his account. After verifying that you really are talking to the real Samson, enter the following two commands:

  sudo pam_tally2 --user=samson --reset

  sudo pam_tally2

Hands-on lab for configuring pam_tally2

- 7. Now that you've seen how this works, open the `/etc/pam.d/login` file for editing. Change the `deny=` parameter from `4` to `100` and save the file. (This will make your configuration a bit more realistic in terms of modern security philosophy.)

Lab 8

Distributed Identity Management

- Install and configure the following tools
  - FreeIPA
  - Adsys

Lab 9

searching for SUID and SGID files

1. Search through the entire filesystem for all the files that have either SUID or SGID set before saving the output to a text file:

sudo find / -type f -perm /6000 -ls > suid_sgid_files.txt

2. Log into any other user account that you have on the system and create a dummy shell script file. Then, set the SUID permission on that file and log back out and back into your own user account:

```
su - desired_user_account
touch some_shell_script.sh
chmod 4755 some_shell_script.sh
ls -l some_shell_script.sh
exit
```

3. Run the find command again, saving the output to a different text file:

sudo find / -type f -perm /6000 -ls > suid_sgid_files_2.txt

4. View the difference between the two files:

diff suid_sgid_files.txt suid_sgid_files_2.txt

Lab 10

searching for SUID and SGID files

For this lab, you'll need to create a perm_demo.txt file with some text of your choice. You'll set the i and a attributes and view the results.

1. Using your preferred text editor, create the perm_demo.txt file with a line of text.

2. View the extended attributes of the file:

lsattr perm_demo.txt

3. Add the a attribute:

sudo chattr +a perm_demo.txt

lsattr perm_demo.txt

searching for SUID and SGID files

4. Try to overwrite and delete the file:
echo "I want to overwrite this file." > perm_demo.txt
sudo echo "I want to overwrite this file." > perm_demo.txt
rm perm_demo.txt
sudo rm perm_demo.txt
5. Now, append something to the file:
echo "I want to append this line to the end of the file." >> perm_demo.txt
6. Remove the a attribute and add the i attribute:
sudo chattr -a perm_demo.txt
lsattr perm_demo.txt
sudo chattr +i perm_demo.txt
lsattr perm_demo.txt
7. Repeat Step 4.

searching for SUID and SGID files

8. Additionally, try to change the filename and create a hard link to the file:

mv perm_demo.txt some_file.txt

sudo mv perm_demo.txt some_file.txt

ln ~/perm_demo.txt ~/some_file.txt

sudo ln ~/perm_demo.txt ~/some_file.txt

9. Now, try to create a symbolic link to the file:

ln -s ~/perm_demo.txt ~/some_file.txt

Lab 11

creating a shared group directory

1. On any virtual machine, create the sales group:

sudo groupadd sales

2. Create the users mimi, mrgray, and mommy, adding them to the sales group as you create the accounts.

On CentOS or AlamaLinux, do this:

sudo useradd -G sales mimi

sudo useradd -G sales mrgray

sudo useradd -G sales mommy

On Ubuntu, do this:

sudo useradd -m -d /home/mimi -s /bin/bash -G sales mimi

sudo useradd -m -d /home/mrgray -s /bin/bash -G sales mrgray

sudo useradd -m -d /home/mommy -s /bin/bash -G sales mommy

creating a shared group directory

3. Assign each user a password.

4. Create the sales directory in the root level of the filesystem. Set proper ownership and permissions, including the SGID and sticky bits:

```
sudo mkdir /sales
sudo chown nobody:sales /sales
sudo chmod 3770 /sales
ls -ld /sales
```

5. Log in as Mimi, and have her create a file:

```
su - mimi
cd /sales
echo "This file belongs to Mimi." > mimi_file.txt
ls –l
```

6. Have Mimi set an ACL on her file, allowing only Mr. Gray to read it. Then, have Mimi log back out:

```
chmod 600 mimi_file.txt
setfacl -m u:mrgray:r mimi_file.txt
getfacl mimi_file.txt
ls -l
exit
```

creating a shared group directory

3. Assign each user a password.

4. Create the sales directory in the root level of the filesystem. Set proper ownership and permissions, including the SGID and sticky bits:

```
sudo mkdir /sales
sudo chown nobody:sales /sales
sudo chmod 3770 /sales
ls -ld /sales
```

5. Log in as Mimi, and have her create a file:

```
su - mimi
cd /sales
echo "This file belongs to Mimi." > mimi_file.txt
ls –l
```

6. Have Mimi set an ACL on her file, allowing only Mr. Gray to read it. Then, have Mimi log back out:

```
chmod 600 mimi_file.txt
setfacl -m u:mrgray:r mimi_file.txt
getfacl mimi_file.txt
ls -l
exit
```

creating a shared group directory

7. Have Mr. Gray log in to see what he can do with Mimi's file. Then, have Mr. Gray create his own file and log back out:

su - mrgray

cd /sales

cat mimi_file.txt

echo "I want to add something to this file." >> mimi_file.txt

echo "Mr. Gray will now create his own file." > mr_gray_file.txt

ls -l

Exit

8. Mommy will now log in and try to wreak havoc by snooping in other users' files and by trying to delete them:

su - mommy

cat mimi_file.txt

cat mr_gray_file.txt

rm -f mimi_file.txt

rm -f mr_gray_file.txt

exi

Lab 12

SELinux type enforcement

- In this lab, you'll install the Apache web server and the appropriate SELinux tools. You'll then view the effects of having the wrong SELinux type assigned to a web content file.
  - Install Apache, along with all the required SELinux tools on CentOS 7:
    - sudo yum install httpd setroubleshoot setools policycoreutils policycoreutils-python
  - On AlmaLinux 8 or 9, use the following command:
    - sudo dnf install httpd setroubleshoot setools policycoreutils policycoreutils-python-utils
  - Activate **setroubleshoot** by restarting the auditd service:
    - sudo service auditd restart
  - Enable and start the Apache service and open port 80 on the firewall:
    - sudo systemctl enable --now httpd
    - sudo firewall-cmd --permanent --add-service=http
    - sudo firewall-cmd --reload

SELinux type enforcement

- In the /var/www/html/ directory, create an index.html file with the following contents:

```
<html>
<head>
<title>SELinux Test Page</title>
</head>
<body>
This is a test of SELinux.
</body>
</html>
```

- View the information about the index.html file:
  - ls -Z index.html
- In your host machine's web browser, navigate to the IP address of the CentOS virtual machine. You should be able to view the page.

SELinux type enforcement

- Induce an SELinux violation by changing the type of the index.html file to something that's incorrect:
  - sudo chcon -t tmp_t index.html
  - ls -Z index.html
- Go back to your host machine's web browser and reload the document. You should now see a **Forbidden** message.
- Use **restorecon** to change the file back to its correct type:
  - sudo restorecon index.html
- Reload the page in your host machine's web browser. You should now be able to view the page.
- End of lab.

Lab 13

SELinux Booleans and ports

- View the ports that SELinux allows the Apache web server daemon to use:
  - sudo semanage port -l | grep 'http’
- Open the /etc/httpd/conf/httpd.conf file in your favorite text editor. Find the line that says Listen 80 and change it to Listen 82. Restart Apache by entering the following:
  - sudo systemctl restart httpd
- View the error message you receive by entering:
  - sudo tail -20 /var/log/messages
- Add port 82 to the list of authorized ports and restart Apache:
  - sudo semanage port -a 82 -t http_port_t -p tcp
  - sudo semanage port -l
  - sudo systemctl restart httpd
- Delete the port that you just added:
  - sudo semanage -d 82 -t http_port_t -p tcp
- Go back into the /etc/httpd/conf/httpd.conf file and change Listen 82 back to Listen 80. Restart the Apache daemon to return to normal operation.

Lab 14

Hands-on lab for basic iptables usage

- Ubuntu 20.04 virtual machine
  - Shut down your Ubuntu virtual machine and create a snapshot. After you boot it back up, look at your iptables rules, or lack thereof, by using this command:
    - sudo iptables –L
  - With a default Ubuntu setup, the Uncomplicated Firewall (ufw) service is already running, albeit with an unactivated firewall configuration. We want to disable it to work directly with iptables. Do that now with this command:
    - sudo systemctl disable --now ufw

Hands-on lab for basic iptables usage

- Ubuntu 20.04 virtual machine
  - Create the rules that you need for a basic firewall, allowing for Secure Shell access, DNS queries and zone transfers, and the proper types of ICMP. Deny everything else:

```
sudo iptables -A INPUT -m conntrack  --ctstate ESTABLISHED,RELATED  -j
ACCEPT

sudo iptables -A INPUT -p tcp --dport ssh -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 53 -j ACCEPT
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT
sudo iptables -A INPUT -m conntrack -p icmp --icmp-type 3 --ctstate
NEW,ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A INPUT -m conntrack -p icmp --icmp-type 11 --ctstate
NEW,ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A INPUT -m conntrack -p icmp --icmp-type 12 --ctstate
NEW,ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A INPUT -j DROP
```

Hands-on lab for basic iptables usage

- Ubuntu 20.04 virtual machine
  - Use this command to view the results:
    - sudo iptables –L
  - You forgot about that loopback interface. Add a rule for it at the top of the list:
    - sudo iptables -I INPUT 1 -i lo -j ACCEPT
  - View the results by using the following two commands. Note the difference between the output of each:
    - sudo iptables -L
    - sudo iptables -L –v
  - Install the iptables-persistent package and choose to save the IPv4 and IPv6 rules when prompted:
    - sudo apt install iptables-persistent
  - Reboot the virtual machine and verify that your rules are still active.
  - Keep this virtual machine; you'll be adding more to it in the next hands-on lab.
  - End of lab

Lab 15

Blocking invalid IPv4 packets

- Use the same virtual machine that you used for the previous lab. You won't replace any of the rules that you already have.
  1. Look at the rules for the filter and the mangle tables. (Note that the -v option shows you statistics about packets that were blocked by the DROP and REJECT rules.) Then, zero out the blocked packets counter:
     - sudo iptables -L -v
     - sudo iptables -t mangle -L -v
     - sudo iptables -Z
     - sudo iptables -t mangle –Z
  2. From either your host machine or another virtual machine, perform the NULL and Windows Nmap scans against the virtual machine:
     - sudo nmap -sN ip_address_of_your_VM
     - sudo nmap -sW ip_address_of_your_VM
  3. Repeat Step 1. You should see a large jump in the number of packets that were blocked by the final DROP rule in the INPUT chain of the filter table:
     - sudo iptables -L -v
     - sudo iptables -t mangle -L -v

Blocking invalid IPv4 packets

- Make the firewall work more efficiently by using the PREROUTING chain of the mangle table to drop invalid packets, such as those that are produced by the two Nmap scans that we just performed. Add the two required rules with the following two commands:
  - sudo iptables -t mangle -A PREROUTING -m conntrack --ctstate INVALID –j DROP
  - sudo iptables -t mangle -A PREROUTING -p tcp ! --syn -m conntrack --ctstate NEW -j DROP

- Save the new configuration to your own home directory. Then, copy the file to its proper location and zero out the blocked packet counters:
  - sudo iptables-save > rules.v4
  - sudo cp rules.v4 /etc/iptables
  - sudo iptables -Z
  - sudo iptables -t mangle -Z

Blocking invalid IPv4 packets

- Perform only the NULL scan against the virtual machine:
  - sudo nmap -sN ip_address_of_your_VM
- Look at the iptables ruleset and observe which rule was triggered by the Nmap scan:
  - sudo iptables -L -v
  - sudo iptables -t mangle -L –v
- This time, perform just the Windows scan against the virtual machine:
  - sudo nmap -sW ip_address_of_your_VM
- Observe which rule was triggered by this scan:
  - sudo iptables -L -v
  - sudo iptables -t mangle -L –v
- End of this lab

Lab 16

Lab for ip6tables

- For this lab, you will use the same Ubuntu virtual machine that you used in the previous iptables labs.
- You will leave the IPv4 firewall setup that's already there as-is and create a new firewall for IPv6.
- View your IPv6 rules, or lack thereof, with this command:
  - sudo ip6tables
- View the new ruleset by using the following command:
  - sudo ip6tables –L
- Next, set up the mangle table rules for blocking invalid packets:
  - sudo ip6tables -t mangle -A PREROUTING -m conntrack --ctstate INVALID –j DROP
  - sudo ip6tables -t mangle -A PREROUTING -p tcp ! --syn -m conntrack --ctstate NEW -j DROP
- Save the new ruleset to a file in your own home directory, and then transfer the rules file to the proper location:
  - sudo ip6tables-save > rules.v6
  - sudo cp rules.v6 /etc/iptables/

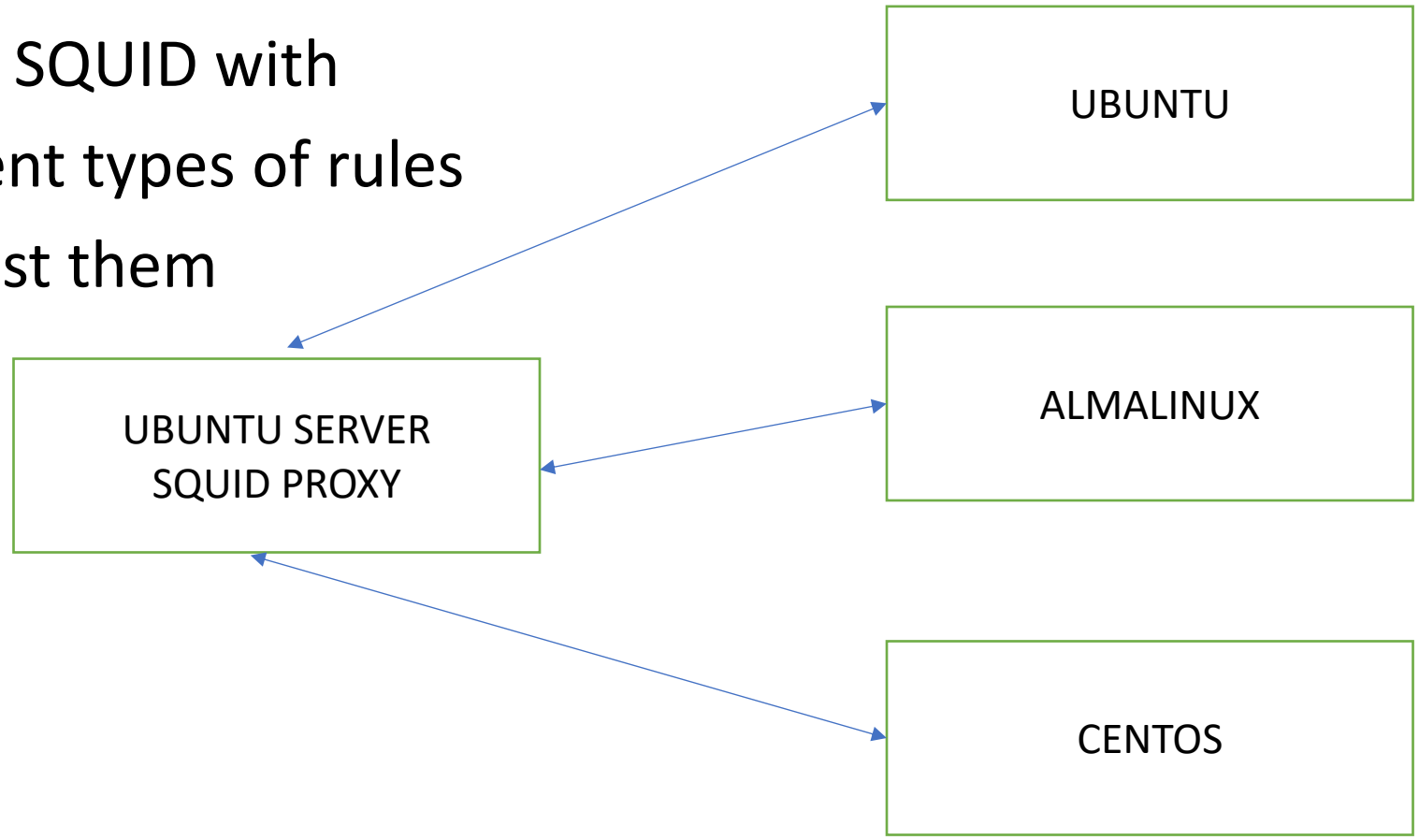
Lab for ip6tables

- Obtain the IPv6 address of the virtual machine with this command:
  - ip a
- On the machine on which you installed Nmap, perform a Windows scan of the virtual machine's IPv6 address. The command will look like this, except with your own IP address:
  - sudo nmap -6 -sW fe80::a00:27ff:fe9f:d923
- On the virtual machine, observe which rule was triggered with this command:
  - sudo ip6tables -t mangle -L –v
  - You should see non-zero numbers for the packet counters for one of the rules.
- On the machine on which you installed Nmap, perform an XMAS scan of the virtual machine's IPv6 address. The command will look like this, except with your own IP address:
  - sudo nmap -6 -sX fe80::a00:27ff:fe9f:d923
- As before, on the virtual machine, observe which rule was triggered by this scan:
  - sudo ip6tables -t mangle -L –v
- Shut down the virtual machine, and restore it from where you started lab 14

Lab 17

ACL SQUID

- Set up SQUID with different types of rules and test them



Lab 18

Firewall and DMZ rules

- Now consider the following set of firewall rules
  - https://www.cs.montana.edu/courses/spring2004/409/topics/nat/firewall.html
  - Describe and explain them
  - Adapt and experiment them in your network
- Now consider the following set of firewall rules
  - https://www.cs.montana.edu/courses/spring2004/409/topics/nat/dmz.html
  - Describe and explain them
  - Adapt and experiment them in your network
- Now consider the following set of firewall rules
  - https://www.cyberciti.biz/tips/wp-content/uploads/2006/06/fw.proxy.txt
  - Describe and explain them
  - Adapt and experiment them in your network

Lab 19

nftables on Ubuntu

- Restore your Ubuntu virtual machine to a clean snapshot to clear out any firewall configurations that you created previously. (Or, if you prefer, start with a new virtual machine.) Disable ufw and verify that no firewall rules are present:
  - sudo systemctl disable —now ufw
  - sudo iptables –L
- You should see no rules listed for nftables. Copy the workstation.nft template over to the /etc/ directory and rename it nftables.conf:
  - sudo cp /usr/share/doc/nftables/examples/syntax/workstation /etc/nftables.conf

nftables on Ubuntu

- Edit the /etc/nftables.conf file to create your new configuration. (Note that due to formatting constraints, I have to break this into three different code blocks.) Make the top portion of the file look like this:

```
#!/usr/sbin/nft -f flush ruleset
table inet filter {
    chain prerouting {
        type filter hook prerouting priority 0;
        ct state invalid counter log prefix "Invalid Packets:  " drop
        tcp flags & (fin|syn|rst|ack) != syn ct state new counter log prefix
"Invalid Packets 2: " drop
}
```

nftables on Ubuntu

- Make the second portion of the file look like this:

```
chain input {
    type filter hook input priority 0;
    # accept any localhost traffic
    iif lo accept
    # accept traffic originated from us
    ct state established,related accept
        # activate the following line to accept common local services
        tcp dport 22 ip saddr { 192.168.0.7, 192.168.0.10 } log prefix "Blocked
SSH packets: " drop
        tcp dport { 22, 53 } ct state new accept
        udp dport 53 ct state new accept
        ct state new,related,established icmp type { destination-unreachable,
time-exceeded, parameter-problem } accept
```

nftables on Ubuntu

- Make the final portion of the file look like this:

```
        ct state new,related,established icmpv6 type { destination-unreachable,
time-exceeded, parameter-problem } accept
        # accept neighbour discovery otherwise Ipv6 connectivity breaks.
        ip6 nexthdr icmpv6 icmpv6 type { nd-neighbor-solicit,  nd-router-
advert, nd-neighbor-advert } accept

# count and drop any other traffic
    counter log prefix "Dropped packet: " drop
    }
}
```

- Save the file and reload nftables:
  - sudo systemctl reload nftables
- View the results:
  - sudo nft list tables
  - sudo nft list tables
  - sudo nft list table inet filter
  - sudo nft list ruleset

nftables on Ubuntu

- From either your host computer or from another virtual machine, do a Windows scan against the Ubuntu virtual machine:
  - sudo nmap -sW ip_address_of_UbuntuVM
- Look at the packet counters to see which blocking rule was triggered. (Hint: It's in the prerouting chain.):
  - sudo nft list ruleset
- This time, do a null scan of the virtual machine:
  - sudo nmap -sN ip_address_of_UbuntuVM
- Finally, look at which rule was triggered this time. (Hint: It's the other one in the prerouting chain.):
  - sudo nft list ruleset
- In the /var/log/kern.log file, search for the Invalid Packets text string to view the messages about the dropped invalid packets.
- End of this lab

Lab 20

Hands-on lab for basic ufw usage

- Shut down your Ubuntu virtual machine and restore the snapshot to get rid of all of the iptables or nftables stuff that you just did. (Or, if you prefer, just start with a fresh virtual machine.)
  - When you've restarted the virtual machine, verify that the iptables rules are now gone. On Ubuntu 20.04 do:
    - sudo iptables –L
  - On Ubuntu 22.04, do:
    - sudo nft list ruleset
  - View the status of ufw. Open port 22/TCP and then enable ufw. Then, view the results:
    - sudo ufw status
    - sudo ufw allow 22/tcp
    - sudo ufw enable
    - sudo ufw status

Hands-on lab for basic ufw usage

- On Ubuntu 20.04, do:
  - sudo iptables -L
  - sudo ip6tables –L
- On Ubuntu 22.04, do:
  - sudo nft list ruleset
- This time, open port 53 for both TCP and UDP:
  - sudo ufw allow 53
  - sudo ufw status
- On Ubuntu 20.04, do:
  - sudo iptables -L
  - sudo ip6tables –L
- On Ubuntu 22.04, do:
  - sudo nft list ruleset

Hands-on lab for basic ufw usage

- cd into the /etc/ufw/ directory. Familiarize yourself with the contents of the files that are there

```
Open the /etc/ufw/before.rules file in your favorite text editor. At
the bottom of the file, below the COMMIT line, add the following code
snippet:



# Mangle table added by Donnie
*mangle
:PREROUTING ACCEPT [0:0]
-A PREROUTING -m conntrack --ctstate INVALID -j DROP

-A PREROUTING -p tcp -m tcp ! --tcp-flags FIN,SYN,RST,ACK SYN -m
conntrack --ctstate NEW -j DROP

COMMIT
```

Hands-on lab for basic ufw usage

- Repeat the previous step for the /etc/ufw/before6.rules file.
- Reload the firewall with this command:
  - sudo ufw reload
- On Ubuntu 20.04, observe the rules by doing:
  - sudo iptables -L
  - sudo iptables -t mangle -L
  - sudo ip6tables -L
  - sudo ip6tables -t mangle –L
- On Ubuntu 22.04, observe the rules by doing:
  - sudo nft list ruleset
- Take a quick look at the ufw status:
  - sudo ufw status
- End of the lab

Lab 21

Hands-on lab – setting up a Dogtag CA

- Dogtag PKI is much simpler to set up, and it has a nice web interface that OpenSSL doesn't have. It's available in the normal repositories of Debian/Ubuntu and RHEL/AlmaLinux, but under different package names.
- In the Debian/Ubuntu repositories, the package name is **dogtag-pki**. In the RHEL/ AlmaLinux repositories, the name is pki-ca.
  - Before we install the Dogtag packages, we need to do a couple of simple chores:
    - Set a Fully Qualified Domain Name (FQDN) on the server.
    - Either create a record in a local DNS server for the Dogtag server, or create an entry for it in its own /etc/hosts file.

Hands-on lab – setting up a Dogtag CA

- You can do this lab on either your AlmaLinux 9 or your Ubuntu 22.04 VM.
- To access the Dogtag dashboard, you'll use a second Linux VM with a desktop environment installed.
  - 1. On your server virtual machine, set an FQDN, substituting your own for the one that I'm using:

    sudo hostnamectl set-hostname donnie-ca.local
  - 2. Edit the /etc/hosts file to add a line like the following:

    192.168.0.53 donnie-ca.local

Hands-on lab – setting up a Dogtag CA

- 3. Use your virtual machine's own IP address and FQDN.
- 4. Next, increase the number of file descriptors that your system can have open at one time. (Otherwise, you'll get a warning message when you run the Directory Server installer.)
- Do that by editing the /etc/security/limits.conf file. At the end of the file, add these two lines:

root hard nofile 4096

root soft nofile 4096

Hands-on lab – setting up a Dogtag CA

- 5. Reboot the machine so that the new hostname and file descriptor limits can take effect.
- 6. Dogtag stores its certificate and user information in an LDAP database. In this step, you'll install the LDAP server package, along with the Dogtag package. For AlmaLinux 9, do this:

  sudo dnf install 389-ds-base pki-ca
- 7. For Ubuntu 22.04, do this:

  sudo apt install 389-ds-base dogtag-pki
- 8. Next, create an LDAP Directory Server (DS) instance by first creating an instance.inf file in the root user's home directory:

  sudo vim /root/instance.inf

Hands-on lab – setting up a Dogtag CA

- 9. Make its contents look something like this, using your own suffix and root_password:

```
# /root/instance.inf
[general]
config_version = 2

[slapd]
root_password = TurkeyLips

[backend-userroot]
sample_entries = yes
suffix = dc=donnie-ca,dc=local
```

- 10. passwords into plain-text: bad practice
- 11. We can now use this instance.inf file, along with the dscreate utility, to create the Directory Server instance:

sudo dscreate from-file /root/instance.inf

Hands-on lab – setting up a Dogtag CA

- 12. Finally, it's time to create the CA:

  sudo pkispawn

- Accept all the defaults until you get to the very end. When it asks Begin Installation?, type Yes. When you get to the Directory Server part, enter the password that you used to create the DS instance in the previous step. Note that you'll be offered the choice to access the LDAP DS instance via a secure port. But since we're setting up LDAP and Dogtag on the same machine, this isn't necessary.
- 13. Ensure that the Dogtag service will automatically start by enabling the pki-tomcatd.target.
- Do that with:

  sudo systemctl enable pki-tomcatd.target

  sudo shutdown -r now

Hands-on lab – setting up a Dogtag CA

- 14. After everything is set up, you will no longer need the instance.inf file that holds your password in plain-text. Get rid of it by doing:

  sudo shred -u -z /root/instance.inf

15. You'll access the Dogtag web interface via port 8443/tcp. On the AlmaLinux machine, open that port like this:

  sudo firewall-cmd --permanent --add-port=8443/tcp

  sudo firewall-cmd –reload

16. On the Ubuntu machine, assuming that you're using the Uncomplicated Firewall, open the port like this:

  sudo ufw allow 8443/tcp

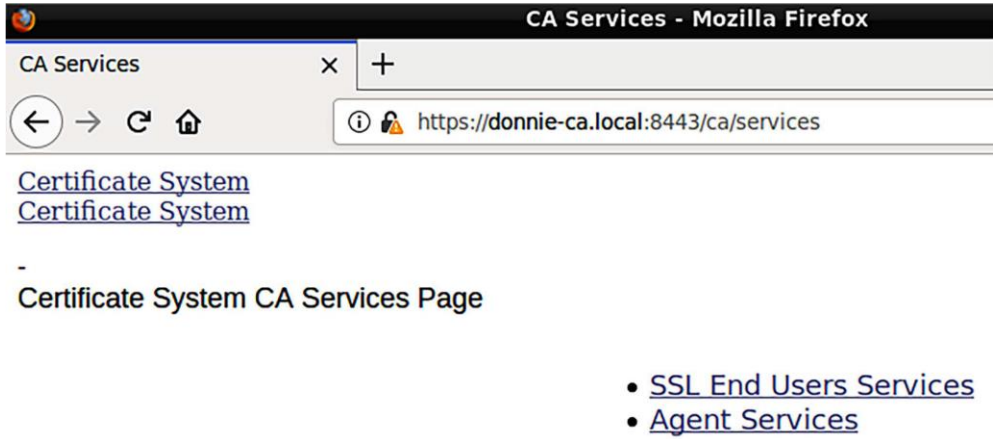
Hands-on lab – setting up a Dogtag CA

- 17. On another Linux virtual machine that has a desktop interface, edit the /etc/hosts file to add the same line that you added to the server hosts file in step 2. Then, open the Firefox web browser on that machine and navigate to the Dogtag dashboard. In keeping with the example in this scenario, the URL would look like this:

  https://donnie-ca.local:8443

Hands-on lab – setting up a Dogtag CA

- You'll receive a warning about the certificate being invalid because it's self-signed.
- That's normal, because every CA has to start with a self-signed certificate, and you haven't yet imported this certificate into your trust store. Temporarily add the exception and continue. (In other words, clear the checkmark from the Add permanently box. You'll see why in the next lab.) Click through the links until you reach this screen:



Lab 22

Hands-on lab – creating and transferring SSH keys

- You will use one virtual machine (VM) as your client, and one VM as the server. Alternatively, if you are using a Windows host machine, you can use Cygwin, PowerShell, or the built-in Windows Bash shell for the client.
- For the server VM, use either Ubuntu 22.04 or CentOS 7.
- On the client machine, create a pair of 384-bit elliptic curve keys. Accept the default filename and location and create a passphrase:
  - ssh-keygen -t ecdsa -b 384
- Observe the keys, taking note of the permissions settings:
  - ls -l ./ssh
- Add your private key to your session keyring. Enter your passphrase when prompted:
  - exec /usr/bin/ssh-agent $SHELL
  - ssh-add

Hands-on lab – creating and transferring SSH keys

- Transfer the public key to the server VM. When prompted, enter the password for your user account on the server VM. (Substitute your own username and IP address in the following command.):

  ssh-copy-id donnie@192.168.0.7

- Log in to the server VM as you normally would:

  ssh donnie@192.168.0.7

- Observe the authorized_keys file that was created on the server VM:

  ls -l .ssh

  cat .ssh/authorized_keys

Hands-on lab – creating and transferring SSH keys

- Log out of the server VM and close the terminal window on the client. Open another terminal window and try to log in to the server again. This time, you should be prompted to enter the passphrase for your private key.
- Log back out of the server VM and add your private key back to the session keyring of your client. Enter the passphrase for your private key when prompted:
  - exec /usr/bin/ssh-agent $SHELL
  - ssh-add
- As long as you keep this terminal window open on your client, you'll be able to log in to the server VM as many times as you want without having to enter a password. However, when you close the terminal window, your private key will be removed from your session keyring.
- You have reached the end of the lab

Lab 22

Hands-on lab — Setting up two-factor authentication on AlmaLinux 8

- For this lab, we assume that you have already installed the Google Authenticator app on your smart phone.
- The Authenticator PAM module is not in any of the repositories for the RHEL 9 distros, but it is in the EPEL repository for the RHEL 8 distros.
- Fire up a fresh AlmaLinux 8 VM to start
  - Install the PAM module like this:
    - sudo dnf install epel-release
    - sudo dnf install google-authenticator qrencode-libs

Hands-on lab — Setting up two-factor authentication on AlmaLinux 8

- Note that you need the **qrencode-libs** package in order to produce a QR code.
- From the GUI terminal of your host machine, use SSH to remotely log in to the AlmaLinux 8 VM. This will allow you to resize the QR code image so that you can take a picture of it with your smart phone. Then, run the google-authenticator:

  google-authenticator -s ~/.ssh/google_authenticator

- Now, we are creating the **google_authenticator** file within the .ssh directory because AlmaLinux is set up to use SELinux. When you try to log in remotely with Authenticator enabled, the SSH daemon will try to write to the google_authenticator file. SELinux prevents SSH from writing to files that are outside of the .ssh directory.

Hands-on lab — Setting up two-factor authentication on AlmaLinux 8

- Follow through on the Authenticator setup, the same as you did in steps 4 through 8 of the Setting up two-factor authentication on Ubuntu 22.04 lab.
- Open the /etc/pam.d/sshd file in your text editor. Add this line to the very bottom of the file:

```
auth required pam_google_authenticator.so secret=/home/${USER}/.ssh/
google_authenticator
```

- Open the /etc/ssh/sshd_config file in your text editor. Find the line that says *#ChallengeResponseAuthentication no* and change it to *ChallengeResponseAuthentication yes*.
- Reload or restart the sshd service:

```
sudo systemctl reload sshd
```

Hands-on lab — Setting up two-factor authentication on AlmaLinux 8

- Set the proper SELinux security context on the **google_authenticator** file that you created:
  - cd
  - sudo restorecon .ssh/google_authenticator

- Log out of the remote session, and try logging back in. This time, you should be prompted to enter a verification code.

Hand-on lab — Using Google Authenticator with key exchange on AlmaLinux 8

- Transfer the public key from your host machine to the AlmaLinux 8 machine as you did in the *Creating and transferring SSH keys* lab.
- In the /etc/ssh/sshd_config file, change the #PasswordAuthentication yes line to PasswordAuthentication no, and reload the SSH configuration. Now, you will only be using key exchange to log in, which will completely bypass the Authenticator. Let's fix things so that you can be using both.
- In the /etc/ssh/sshd_config file, add the following line just beneath the PasswordAuthentication no line:

```
AuthenticationMethods publickey,password publickey,keyboard-interactive
```

Hand-on lab — Using Google Authenticator with key exchange on AlmaLinux 8

- After reloading the SSH configuration, you will have three-factor authentication, because you will need to enter both your password and a verification code along with the key exchange.

- If desired, you can easily disable the password prompt and just use the key exchange and verification code. In the /etc/pam.d/sshd file, find the auth substack password-auth line at the top of the file, and change it to #auth substack password-auth.

- That’s all there is to Google Authenticator.